Rockchip Android 16.0 SSI SDK Developer Guide

ID: RK-KF-YF-E67

Release Version: V1.1.0

Release Date: 2025-10-13

Security Level: □Secret □Internal □Public ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”)DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS,MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip's registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2025. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document introduces the development of the Rockchip Android16 SDK, assisting developers in getting started quickly.

Intended Audience

This document (this guide) is mainly intended for:

Technical Support Engineer

Software Development Engineer

Revision History

Version	Author	Date	Change Description
V1.0.0	Wu Liangqing	2025-7-24	Released the SSI SDK of the RKR1 version supporting RK3576/RK3562
V1.1.0	Wu Liangqing	2025-10-13	Released the SSI SDK of the RKR3 version supporting RK3588 series/RK3568 series/RK356 series/RK3326 series/PX30 series/RK3399

Document Issue Feedback: wlq@rock-chips.com

Contents

Rockchip Android 16.0 SSI SDK Developer Guide

1. Terminology
2. Rockchip Android 16.0 SSI SDK Introduction
3. Rockchip Android 16.0 SSI SDK Supported Chips
4. Rockchip Android 16.0 SSI SDK Code Download and Compilation
    - 4.1 Code Download
        - 4.1.1 Download Address
        - 4.1.2 Server Image Download
    - 4.2 Setting up your own repository code server
        - 4.2.1 Environment
        - 4.2.2 Gitolite Setup
            - 4.2.2.1 Server-side Operations
            - 4.2.2.2 Client Operations
        - 4.2.3 Repository Mirror Setup
            - 4.2.3.1 Server-side Operations
            - 4.2.3.2 Client Operations
        - 4.2.4 Client Operations
    - 4.3 Code Management
        - 4.3.1 Switch your code branch
        - 4.3.2 Code Modification Submission
        - 4.3.3 Synchronizing RK's Code
    - 4.4 Kernel Code Path Introduction
    - 4.5 SSI SDK Compilation
        - 4.5.1 Compilation Conditions
        - 4.5.2 Compilation Instructions
        - 4.5.3 Build Tool rkbuild
        - 4.5.4 Compilation Steps
        - 4.5.5 Firmware Description
    - 4.6 Firmware Flashing
        - 4.6.1 Firmware Flashing Tool
    - 4.7 Generic Kernel Image (GKI)
    - 4.8 Flashing Dynamic Partitions with Fastboot
5. Using DTBO Features
6. Modifying the fstab File
7. Modifying parameter.txt
8. Android14 SDK
9. Appendix A Compiling and Development Environment Setup
    - 9.1 Initializing a Build Environment
    - 9.2 Choosing a Branch
    - 9.3 Setting up a Linux build environment
    - 9.4 Installing the JDK
    - 9.5 Configuring USB Access
10. Appendix B SSH Public Key Operation Instructions
    - 10.1 Appendix B-1 SSH public key generation
    - 10.2 Appendix B-2 Use key-chain to manage the key
    - 10.3 Appendix B-3 Multiple devices use the same SSH public key
    - 10.4 Appendix B-4 Switching different SSH public keys on a single device
    - 10.5 Appendix B-5 Key Authority Management
    - 10.6 Appendix B-6 Git Permission Application Instructions

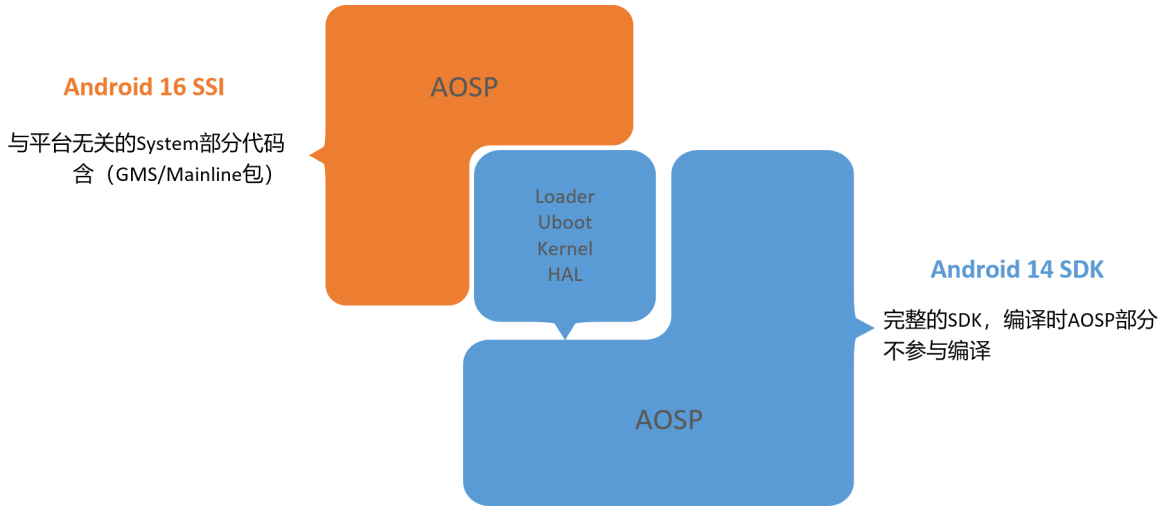
1. Terminology

- SSI Shared System Image

2. Rockchip Android 16.0 SSI SDK Introduction

- Rockchip Android 16.0 SSI SDK only releases aosp code, and after compilation it only generates system-related images such as system.img, system_ext.img, product.img, which are identical to Google's GSI firmware. Other parts of the code need to be compiled using the Android 14 SDK, so Rockchip Android 16.0 SSI SDK needs to be used in conjunction with the Android 14 SDK.

-- The architecture diagram of the complete SDK composed of Rockchip Android 16.0 SSI SDK and Android14 SDK is as follows:



- Rockchip Android 16.0 SSI SDK supports all chip platforms (requires Android 14 update support), i.e., the same system.img can be compatible with all supported chip platforms such as RK3576/RK3562, etc.
- The SSI-based compilation method differs from other SDKs. We have redeveloped a compilation toolchain named rkbuild, which is detailed in the compilation section below.

3. Rockchip Android 16.0 SSI SDK Supported Chips

The SSI system firmware supports all chips. Android14 SDK needs to be updated to the corresponding version. The support status of Android14 SDK versions for Android16 SSI is as follows:

Chip Platform	Supported	Android14 SDK Version	Android16 SSI SDK Version
RK3562	Yes	Android14 RKR7	Android16 RKR1
RK3576 Series	Yes	Android14 RKR7	Android16 RKR2
RK3588 Series	Yes	Android14 RKR8	Android16 RKR3
RK3568 Series	Yes	Android14 RKR8	Android16 RKR3
RK3566 Series	Yes	Android14 RKR8	Android16 RKR3
RK3326 Series	Yes	Android14 RKR8	Android16 RKR3
PX30 Series	Yes	Android14 RKR8	Android16 RKR3
RK3399 Series	Yes	Android14 RKR8	Android16 RKR3

4. Rockchip Android 16.0 SSI SDK Code Download and Compilation

4.1 Code Download

4.1.1 Download Address

```
repo init --repo-url https://gerrit.rock-chips.com:8443/repo-release/tools/repo
-u https://gerrit.rock-chips.com:8443/Android_16/manifests -b master -m
A16_SSI.xml
```

If the intention is to obtain GMS/EDLA certification and the Express version of the SDK is being applied for (requires a Google MADA agreement to apply), then the download link is as follows:

```
repo init --repo-url https://gerrit.rock-chips.com:8443/repo-release/tools/repo
-u https://gerrit.rock-chips.com:8443/Android_16/manifests -b master -m
A16_SSI_Express.xml
```

4.1.2 Server Image Download

```
repo init --repo-url https://gerrit.rock-chips.com:8443/repo-release/tools/repo
-u https://gerrit.rock-chips.com:8443/Android_16/manifests -b master -m
A16_SSI.xml --mirror
```

If the intention is to obtain GMS/EDLA certification and the Express version of the SDK is being applied for (requires a Google MADA agreement to apply), then the download link is as follows:

```
repo init --repo-url https://gerrit.rock-chips.com:8443/repo-release/tools/repo
-u https://gerrit.rock-chips.com:8443/Android_16/manifests -b master -m
A16_SSI_Express.xml --mirror
```

Note: repo is a script written by Google using Python to invoke git, primarily used for downloading and managing software repositories for Android projects. The download address is as follows:

```
git clone https://gerrit.rock-chips.com:8443/repo-release/tools/repo
```

To facilitate customers' quick access to the SDK source code, Rockchip technical windows typically provide the corresponding version's initial SDK package. Taking `Rockchip_Android16.0_SDK_RELEASE.tar.gz.*` as an example, after copying this initial package, the source code can be checked out using the following commands:

```
mkdir Rockchip_Android16.0_SDK_RELEASE
cat Rockchip_Android16.0_SDK_RELEASE.tar.gz* | tar -zx -C
Rockchip_Android16.0_SDK_RELEASE
cd Rockchip_Android16.0_SDK_RELEASE
.repo/repo/repo sync -l
.repo/repo/repo sync -c
```

4.2 Setting up your own repository code server

4.2.1 Environment

Install `openssh-server` for remote login, `git` for version control, and `keychain` as a public-private key management tool.

```
sudo apt-get install openssh-server git keychain
```

4.2.2 Gitolite Setup

4.2.2.1 Server-side Operations

(Taking the server address: 10.10.10.206 as an example)

1. Create a Git account:

```
sudo adduser --system --shell /bin/bash --group git
sudo passwd git
```

2. Log in to the server as the "git" user.
3. Ensure that the "~/.ssh/authorized_keys" file is empty or does not exist.
4. Copy the server administrator's public key to "~/YourName.pub"
5. Download the gitolite source code

```
git clone https://github.com/sitaramc/gitolite.git
```

6. Create a bin directory in the Git user directory.

```
mkdir -p ~/bin
```

7. Execute the following commands to install gitolite. The installation methods vary depending on the version. Please refer to the documentation in the source code:

```
gitolite/install -to ~/bin
```

8. Setting up the Administrator.

```
~/bin/gitolite setup -pk YourName.pub
```

4.2.2.2 Client Operations

1. Clone the server's gitolite administrative repository:

```
git clone ssh://git@10.10.10.206/gitolite-admin.git
```

2. Add the user's public key to the gitolite directory.

```
cp username.pub keydir/username.pub
```

3. Add Administrator User.

```
vi conf/gitolite.conf
@admin = admin1 admin2 admin3
repo gitolite-admin
RW+     =   @admin
```

4.2.3 Repository Mirror Setup

4.2.3.1 Server-side Operations

1. Log in to the server using the Git account.
2. Download the repo tool in the root directory.

```
git clone https://gerrit.rock-chips.com:8443/repo-release/tools/repo
```

3. Create a new directory named RK_Android16_mirror.

```
mkdir RK_Android16_mirror
```

4. Enter the RK_Android16_mirror directory.

```
cd RK_Android16_mirror
```

5. Download RK Android16 SDK image.

```
repo init --repo-url https://gerrit.rock-chips.com:8443/repo-release/tools/repo
-u https://gerrit.rock-chips.com:8443/Android_16/manifests -m A16_SSI.xml --
mirror
```

6. Create Repository Group Permissions.

```
.repo/repo/repo list -n > Android_16.conf
sed -i 's/^/@Android_16 = RK_Android16_mirror\/&/g' Android_16.conf
```

4.2.3.2 Client Operations

1. Copy the server-side Android_16.conf to the client-side ·gitolite-admin/conf/· directory.
2. Add Group Permissions.

```
vi conf/Android_16.conf
@usergroup = user1 user2 user3
repo @Android_16
R   = @usergroup
RW+ = @admin
```

```
vi conf/gitolite.conf
include "Android_16.conf"
```

3. Create your own manifests repository.

```
vi conf/Android_16.conf
@Android_16 = Android_T/manifests_xxx
```

4.2.4 Client Operations

1. Clone the manifests_xxx repository on the client.

Clone the manifests_xxx.git repository on other client machines.

```
git clone ssh://git@10.10.10.206/Android_16/manifests_xxx.git
```

2. Download the original manifests repository on the client.

```
git clone ssh://git@10.10.10.206/Android_16/manifests.git
```

3. Commit the manifest.xml file to the newly created manifest_xxx repository.

Copy the files from the original manifests directory to the manifests_xxx directory.

```
cd manifests_xxx
cp -rf manifests/*.xml manifests_xxx/
```

View Copied Files:

```
git status

 Android16.xml
 Android16_Express.xml
 default.xml
 include/rk3562_repository.xml
 include/rk_checkout_from_aosp.xml
 include/rk_modules_repository.xml
 remote.xml
 remove_u.xml
 remove_unused.xml
```

Local Commit:

```
git add -A
git commit -m "init xxx"
```

push to the remote branch:

```
git push origin master:master
```

4. Creating Your Own Code Download Link

Download the repo tool in the root directory

```
git clone https://gerrit.rock-chips.com:8443/repo-release/tools/repo
```

After following the above steps, the download link for your code is as follows:

```
mkdir Android16
cd Android16
~/repo/repo init -u ssh://git@10.10.10.206/Android_16/manifests_xxx.git -m
Android16.xml
```

Among them:

`//10.10.10.206` is your server address.

By following the above steps, you can complete the setup of your own repository server and share the link to your code repository with your colleagues to collaborate on projects.

4.3 Code Management

After setting up the code server through the above steps, most code repositories use the default branch of RK. If a repository requires modifying its own code, refer to the following steps for the operation.

4.3.1 Switch your code branch

1. Navigate to the code repository that requires modification, using the kernel directory as an example for illustration.

```
cd kernel-6.1
```

2. Switch to a local branch.

```
git checkout remotes/m/master -b xxx_branch
```

3. push `xxx_branch` branch to the remote server.

```
git push rk29 xxx_branch:xxx_branch
```

Among them, `rk29` is a remote that can be auto-completed using the Tab key.

4. Navigate to the `.repo/manifests` directory to modify the branch specified in the manifest.

Navigate to the `.repo/manifests` directory using `grep kernel` to locate the position of the manifest corresponding to the kernel repository.

```
cd .repo/manifests
```

```
--- a/include/rk_modules_repository.xml
+++ b/include/rk_modules_repository.xml
@@ -10,7 +10,7 @@
   <project path="hardware/rockchip/libgraphicpolicy"
name="rk/hardware/rk29/libgraphicpolicy" remote="rk"
revision="refs/tags/android-1s.0-mid-rkr1" />
   <project path="hardware/rockchip/libhwjpeg" name="rk/hardware/rk29/libhwjpeg"
remote="rk" revision="refs/tags/android-16.0-mid-rkr1"/>
   <project path="u-boot" name="rk/u-boot" remote="rk"
revision="refs/tags/android-16.0-mid-rkr1"/>

-  <project path="kernel" name="rk/kernel" remote="rk29"
revision="refs/tags/android-16.0-mid-rkr1"/>

+  <project path="kernel" name="rk/kernel" remote="rk29" revision="xxx_branch"/>

   <project path="bootable/recovery/rkupdate"
name="platform/bootable/recovery/rk_update" remote="rk"
revision="refs/tags/android-16.0-mid-rkr1"/>
   <project path="bootable/recovery/rkutility"
name="platform/bootable/recovery/rk_utility" remote="rk"
revision="refs/tags/android-16.0-mid-rkr1"/>
```

5. Commit the modified manifest to the remote branch.

```
git add include/rk_modules_repository.xml
git commit -m "change kernel branch on xxx_branch"
git push origin default:master
```

After submitting the manifests repository, other colleagues can synchronize the kernel code from your own branch.

4.3.2 Code Modification Submission

After completing the branch switch as outlined in the steps above, you can commit your changes on your own branch. Directly push the commits to the xxx_branch.

4.3.3 Synchronizing RK's Code

1. Synchronizing the RK code requires performing a sync operation on the server-side.

```
cd RK_Android16_mirror
```

```
.repo/repo/repo sync -c
```

2. Client merges RK modifications to manifests.

- Download RK original manifests repository.

```
git clone //10.10.10.206/wlq/test/manifests.git
```

Use a comparison tool to compare manifests (RK original) and manifests_xxx (own), and merge the modified differences from RK into your own repository (primarily modifying tags, addition/removal of repositories, etc.).

- After verification, push the changes to manifests_xxx.

This step also confirms which repositories you have modified, and the merging of modified repositories will be performed in the next step.

3. Directories with their own branched copies require manual merging of RK's changes into their respective branches.

Take kernel as an example:

- View the current remote branch being pointed to

```
wlq@wlq:~/home1/test2/kernel-6.1$ git branch -av
* android-11.0-mid-rkr7   0bde59fad73a ARM: configs: rockchip_defconfig enable
ION_CMA_HEAP
  xxx_branch              0bde59fad73a ARM: configs: rockchip_defconfig enable
ION_CMA_HEAP
  remotes/m/master        -> rk29/xxx_branch
  remotes/rk29/xxx_branch 0bde59fad73a ARM: configs: rockchip_defconfig enable
ION_CMA_HEAP
```

It can be seen that the current reference is pointing to: `remotes/m/master        -> rk29/xxx_branch`

- Create a local branch (switch from your own remote branch)

```
git checkout remotes/m/xxx_branch -b local_xxx_branch
```

- Confirm the latest TAG released by RK currently.

```
wlq@wlq:~/home1/test2/kernel$ git tag | grep rkr
android-10.0-mid-rkr1
```

```
android-10.0-mid-rkr10
android-10.0-mid-rkr11
android-10.0-mid-rkr13
android-10.0-mid-rkr2
android-10.0-mid-rkr3
android-10.0-mid-rkr4
android-10.0-mid-rkr5
android-10.0-mid-rkr6
android-10.0-mid-rkr7
android-10.0-mid-rkr8
android-10.0-mid-rkr9
android-11.0-ebook-rkr1
android-11.0-ebook-rkr2
android-11.0-ebook-rkr3
android-11.0-ebook-rkr4
android-11.0-ebook-rkr5
android-11.0-ebook-rkr6
android-11.0-mid-rkr1
android-11.0-mid-rkr2
android-11.0-mid-rkr3
android-11.0-mid-rkr4
android-11.0-mid-rkr4.1
android-11.0-mid-rkr5
android-11.0-mid-rkr6
android-11.0-mid-rkr7
android-11.0-mid-rkr7-prev
android-11.0-mid-rkr8
android-16.0-mid-rkr1
```

It can be observed that the current latest tag for Android16 is `android-16.0-mid-rkr1`

- Merge `android-16.0-mid-rkr1` into the local branch

```
git merge android-16.0-mid-rkr1
```

Check if there are any conflicts. If there are conflicts, resolve them first. If there are no conflicts, proceed to the next step.

- Push the merged code to the remote branch

```
git push rk29 local_xxx_branch:xxx_branch
```

- Other split directories can be merged and submitted in this manner.

4.4 Kernel Code Path Introduction

Android16 supports the 6.1 version of the kernel. The kernel source code is located in the kernel-6.1 directory of the project.

4.5 SSI SDK Compilation

4.5.1 Compilation Conditions

Android 14 SDK is required, and the Android 14 SDK version must be RKR7 or higher. Compilation requires linking to the Android 14 SDK. If the Android 14 SDK is not available, it needs to be downloaded first. The download link is as follows:

- RK3576 Andriod14 SDK

```
repo init --repo-url https://gerrit.rock-chips.com:8443/repo-release/tools/repo
-u https://gerrit.rock-chips.com:8443/Android_U/manifests -b rk3576 -m
Android14.xml
```

- Android 14 SDK for other chips

```
repo init --repo-url https://gerrit.rock-chips.com:8443/repo-release/tools/repo
-u https://gerrit.rock-chips.com:8443/Android_U/manifests -b master -m
Android14.xml
```

4.5.2 Compilation Instructions

SSI compilation is divided into system (Android 16 AOSP) and vendor (Android 14 non-AOSP parts), which are compiled separately and then merged into a complete firmware image using tools. The compilation is performed using the rkbuild tool.

4.5.3 Build Tool rkbuild

- Executing the `source build/envsetup.sh` command in the Android 16 SSI SDK creates rkbuild-related tool links.

```
Android16_SSI_SDK$ source build/envsetup.sh

Android16_SSI_SDK$ rkbuild
rkbuild     rkbuild_build     rkbuild_merge     rkbuild_mkupdate     rkbuild_pathmod
   rkbuild_reset     rkbuild_setup     rkbuild_status     rkbuild_usage
```

- Source code location of rkbuild:

```
device/rockchip/rockchip_generic/vendorsetup.sh
```

- rkbuild Usage Instructions

```
@sys2206:~/d1_Android16_ssi$ rkbuild_usage
Usage: -bash { setup | reset | status | pathmod | build | help }
- setup { -d <kernel_dts> -l <vendor_lunch> -f <build_number> -p
<vendor_sdk_path> }
    Setup the Android_VENDOR_SDK software link
- reset
    Reset the Android_VENDOR_SDK software link
- status
    Query the Android_VENDOR_SDK software link status
- pathmod
    Print the source directory corresponding to the module name
    pathmod libfoo[.vendor], libfoo.vendor will look for libfoo in the GRF SDK
```

```
- merge
    Merge images from target-files
- mkupdate
    Create update.img
- build { uboot | kernel | ssi | vendor | all }
    - uboot: Build the uboot in Android_VENDOR_SDK
    - kernel: Build the kernel modules in Android_VENDOR_SDK
    - ssi: Build the system SDK
    - vendor: Build the GRF SDK
    - all: Build the uboot & kernel & ssi & vendor, then pack the images to
update.img
- help
```

4.5.4 Compilation Steps

- Verify that the Android16 SSI SDK and Android14 SDK have been downloaded.
- Navigate to the Android16 SSI SDK root directory to configure the build environment.

```
@sys2206:~/d1_Android16_ssi$ source build/envsetup.sh
```

- Compile SSI's system

The corresponding device product directory for SSI is:

```
device/rockchip/rockchip_generic/
```

The corresponding product lunch is defined in:

```
device/rockchip/rockchip_generic/AndroidProducts.mk

 21 COMMON_LUNCH_CHOICES := \
 22     ssi_16_arm64-bp2a-userdebug \
 23     ssi_16_arm64-bp2a-user \
 24     ssi_16_arm64_go-bp2a-userdebug \
 25     ssi_16_arm64_go-bp2a-user
```

The ssi_16_arm64 and ssi_16_arm64_go configurations are supported.

Select the corresponding lunch.

```
@sys2206:~/d1_Android16_ssi$ lunch ssi_16_arm64-bp2a-userdebug

============================================
PLATFORM_VERSION_CODENAME=REL
PLATFORM_VERSION=16
TARGET_PRODUCT=ssi_16_arm64
TARGET_BUILD_VARIANT=userdebug
TARGET_ARCH=arm64
TARGET_ARCH_VARIANT=armv8-a
TARGET_CPU_VARIANT=generic
HOST_OS=linux
HOST_OS_EXTRA=Linux-5.15.0-76-generic-x86_64-Ubuntu-20.04.6-LTS
HOST_CROSS_OS=windows
```

```
BUILD_ID=BP2A.250605.031.A3
OUT_DIR=out
SOONG_ONLY=false
============================================

```````````````````````````````````````````````````````````````````````````````~
  Wondering whether to use user, userdebug or eng?

  user   The builds that ship to users. Reduced debugability.
  userdebug   High fidelity to user builds but with some debugging options
              enabled. Best suited for performance testing or day-to-day use
              with debugging enabled.
  eng   More debugging options enabled and faster build times, but
              runtime performance tradeoffs. Best suited for day-to-day
              local development when not doing performance testing.
```````````````````````````````````````````````````````````````````````````````~
```

Start compiling system

- Method 1: Using make to compile

```
@sys2206:~/d1_Android16_ssi$ make dist -j32
```

- Method 2, using the rkbuild script to compile

```
@sys2206:~/d1_Android16_ssi$ rkbuild_build ssi
```

- Compile the full firmware

1). Linking Android14 SDK and configuring build parameters

```
@sys2206:~/d1_Android16_ssi$ rkbuild_setup
Please enter the directory of Android_VENDOR_SDK (e.g.,
/home/hcq/4_Android14_0409): /home/wlq/b2_Android14_rk3576/  # Enter the
Android14 SDK path here

[WARN] Android_VENDOR_SDK optional lunch targets are as follows:

You're building on Linux

Lunch menu .. Here are the common combinations:
    1. rk3562_u-user
    2. rk3562_u-userdebug
    3. rk3562_ugo-user
    4. rk3562_ugo-userdebug
    5. rk3576_u-user
    6. rk3576_u-userdebug
    7. rk3576s_u-user
    8. rk3576s_u-userdebug

Please enter the lunch option for Android_VENDOR_SDK (e.g., rk3588_u-user):
rk3576_u-serdebug   # Select the lunch option corresponding to Android14 here
```

```
Specify build number? (Press Enter to auto-generate, or directly input the
desired build number):    # Enter the build number here (optional, press Enter
to skip)

device/rockchip/rockchip_generic/configs/gms.mk:10: warning: Please set client
id with your own MADA ID! now is android-rockchip
device/rockchip/rockchip_generic/modules/gms.mk:27: warning: Final select gms-
mandatory_64bit_only and mainline_modules with android-rockchip
.
.
build/make/core/board_config.mk:241: warning: ------------------------->>>>>
TARGET_DEVICE_DIR is device/rockchip/rockchip_generic/ssi_16_arm64
~/b2_Android14_rk3576 ~/d1_Android16_develop

============================================
PLATFORM_VERSION_CODENAME=REL
PLATFORM_VERSION=14
PRODUCT_INCLUDE_TAGS=com.android.mainline
TARGET_PRODUCT=rk3576_u
TARGET_BUILD_VARIANT=userdebug
TARGET_ARCH=arm64
TARGET_ARCH_VARIANT=armv8-a
TARGET_CPU_VARIANT=generic
HOST_OS=linux
HOST_OS_EXTRA=Linux-5.15.0-76-generic-x86_64-Ubuntu-20.04.6-LTS
HOST_CROSS_OS=windows
BUILD_ID=UQ1A.240205.004.B1
OUT_DIR=out
============================================
~/d1_Android16_ssi
Current PRODUCT_KERNEL_DTS=rk3576-evb1-v10, #If you need to modify the DTS,
please enter, press Enter to use the default value: 📄 Parsing module path:

/home/wlq/d1_Android16_ssi/.repo/manifests/include/rk_modules_repository.xml
[WARN] Soft link cache file will be saved in:
/home/wlq/d1_Android16_ssi/.symlinks_cache
[WARN] ⚠ The target already exists but is not a soft link, please check whether
there are any local modifications not yet committed:
/home/wlq/d1_Android16_ssi/bootable/recovery/pcba_core
/home/wlq/d1_Android16_ssi/vendor/rockchip/hardware/interfaces/neuralnetworks
[INFO] 🔄 Removing existing soft link:
/home/wlq/d1_Android16_ssi/vendor/rockchip/hardware/interfaces/rksoundsetting
{
"Android_SYSTEM_SDK": "/home/wlq/d1_Android16_develop",
"Android_VENDOR_SDK": "/home/wlq/b2_Android14_rk3576",
"system_lunch": "ssi_16_arm64-bp2a-userdebug",
"system_product": "ssi_16_arm64",
"system_variant": "userdebug",
"system_release": "bp2a",
"vendor_lunch": "rk3576_u-userdebug",
"vendor_product": "rk3576_u",
"vendor_variant": "userdebug",
"platform_version": "Android16",
"product_api_level": "36",
"build_id": "BP2A.250605.031.A3",
"super_partition_size": "5872025600",
"build_with_google_market": "true",
"build_with_google_gms_express": "false",
```

```
"platform_soc": "rk3576",
"uboot_config": "rk3576_defconfig gki.config",
"kernel_config": "gki_defconfig rockchip_gki.config bifrost_gpu_gki.config
pcie_wifi.config",
"kernel_dts": "rk3576-evb1-v10",
"clang_version": "clang-r487747c",
"kernel_path": "/home/wlq/b2_Android14_rk3576/kernel-6.1",
"build_number": "202507",
"symlink_cache": "/home/wlq/d1_Android16_ssi/.symlinks_cache",
"grf_output": "/home/wlq/d1_Android16_ssi/IMAGE/GRF/ssi_16_arm64",
"dist_direc": "/home/wlq/d1_Android16_ssi/IMAGE/GRF/ssi_16_arm64/dist",
"images_dir": "/home/wlq/d1_Android16_ssi/IMAGE/GRF/ssi_16_arm64/rk3576_u-
userdebug"
}
```

The above compilation steps can be completed using shortcut commands such as:

```
rkbuild setup -l {lunch_name} -d {dts_name} -p /home/wlq/b2_Android14_rk3576 -f
202506
```

Among them:

- -l is used to specify the lunch name for Android14
- -d specifies the kernel's DTS
- -p specifies the path to the Android 14 SDK
- -f specifies the build number, which can be omitted

2). Start compilation

Full Compilation

```
@sys2206:~/d1_Android16_ssi$ rkbuild_build all
```

Partial Compilation

```
- build { uboot | kernel | ssi | vendor | all }
    - uboot: Build the uboot in Android_VENDOR_SDK
    - kernel: Build the kernel modules in Android_VENDOR_SDK
    - ssi: Build the system SDK
    - vendor: Build the GRF SDK
    - all: Build the uboot & kernel & ssi & vendor, then pack the images to
update.img
```

For partial compilation, need to perform merge operation after compilation.

```
@sys2206:~/d1_Android16_ssi$ rkbuild_merge
```

3). Compiled Firmware Path

```
@sys2206:~/d1_Android16_ssi/rockdev/Image-rk3576_u$
```

### 4.5.5 Firmware Description

| Firmware | Source Files | Description |
| --- | --- | --- |
| baseparameter.img | Android14 SDK | Display parameter information |
| boot.img | Android14 SDK | The GKI version is released by Google and includes the kernel core components. The non-GKI version is compiled from the kernel-6.1 source code and includes ramdisk, kernel, dtb |
| boot-debug.img | Android14 SDK | The difference from boot.img is that user firmware can flash this boot.img for root access operations |
| dtbo.img | Android14 SDK | Device Tree Overlays Refer to the dtbo section below for details. |
| init_boot.img | Android14 SDK | Available only in GKI versions, contains ramdisk content |
| config.cfg | Android14 SDK | Configuration file for the flashing tool, which can be directly imported into the flashing tool to display the options that need to be flashed. |
| MiniLoaderAll.bin | Android14 SDK | Primary loader, includes ddr, spl |
| misc.img | Android14 SDK | Contains recovery-wipe boot identifier information, and will enter recovery after flashing |
| parameter.txt | Android14 SDK | contains partition information |
| pcba_small_misc.img | Android14 SDK | Contains PCBA boot identifier information, enters simplified PCBA mode after flashing. |
| pcba_whole_misc.img | Android14 SDK | contains the PCBA boot identifier information, after flashing it will enter the full version PCBA mode |
| recovery.img | Android 14 SDK | only present in non-GKI versions, includes recovery-ramdis, kernel, and dtb |
| super.img | Android16 SDK + Android14 SDK | contains the contents of the odm, product, vendor, system, and system_ext partitions, where the system and system_ext partitions are compiled from the Android16 SDK |
| uboot.img | Android14 SDK | Compiled from the u-boot source code |
| vbmeta.img | Android14 SDK | contains AVB verification information for AVB verification |
| update.img | Android16 SDK + Android14 SDK | Contains the img files mentioned above, can be used by tools to directly flash the entire firmware package |

| Firmware | Source Files | Description |
| --- | --- | --- |
| ssi_16_arm64-images-merged-fastboot.zip | Android16 SDK + Android14 SDK | A firmware package merged from compiled materials of Android16 and Android14, in a format that can be flashed using the fastboot command |
| ssi_16_arm64-ota-merged.zip | Android16 SDK + Android14 SDK | Combined from the compiled material packages of Android16 and Andriod14, the merged complete OTA update package used for OTA updates |
| ssi_16_arm64-target_files-merged.zip | Android16 SDK + Android14 SDK | Merged from compiled resources of Android16 and Android14, the merged resource package can be used to create OTA differential packages |

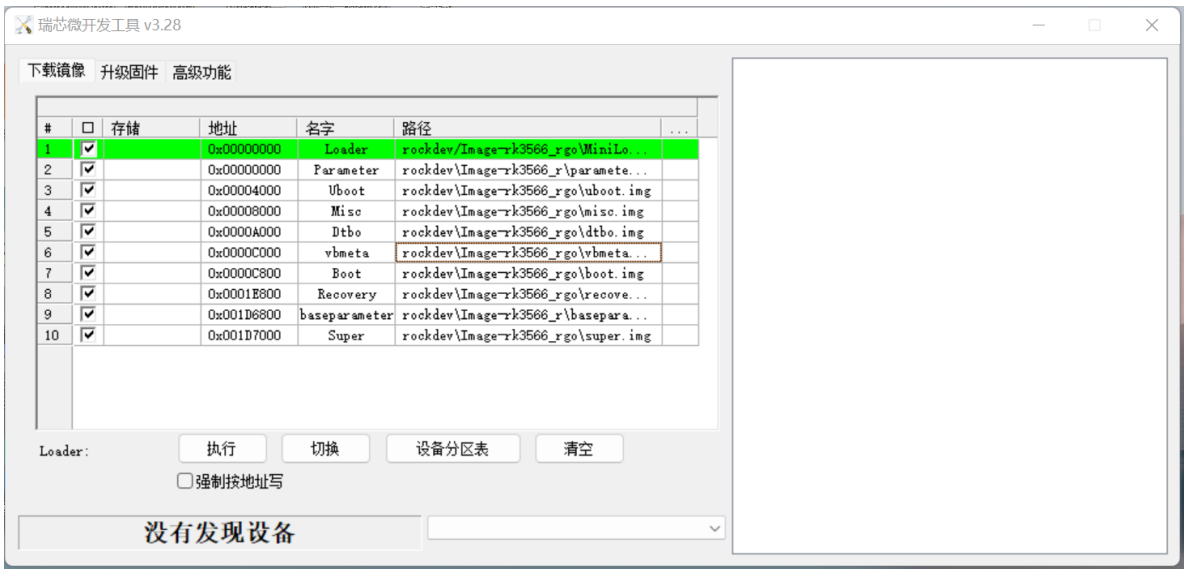
# 4.6 Firmware Flashing

## 4.6.1 Firmware Flashing Tool

The USB driver DriverAssitant for Android16 needs to be updated to version V5.1.1. Refer to the Tools section below for the update procedure.

Windows Flashing Tool: (The tool is constantly updated, please synchronize and update timely)

```
RKTools/windows/AndroidTool/AndroidTool_Release_v3.28
```




```
RKTools/linux/Linux_Upgrade_Tool/Linux_Upgrade_Tool_v2.30
```

In the following section on tool description, detailed explanations are provided.

The compiled unpacked firmware and `update.img` can be used for flashing.

## 4.7 Generic Kernel Image (GKI)

Products that pass GMS and EDLA certification for Android 14 are required to use GKI for the kernel. For specific configuration and compilation of GKI, refer to the document RKDocs/android/Rockchip_Developer_Guide_Android14_GKI_CN.pdf

## 4.8 Flashing Dynamic Partitions with Fastboot

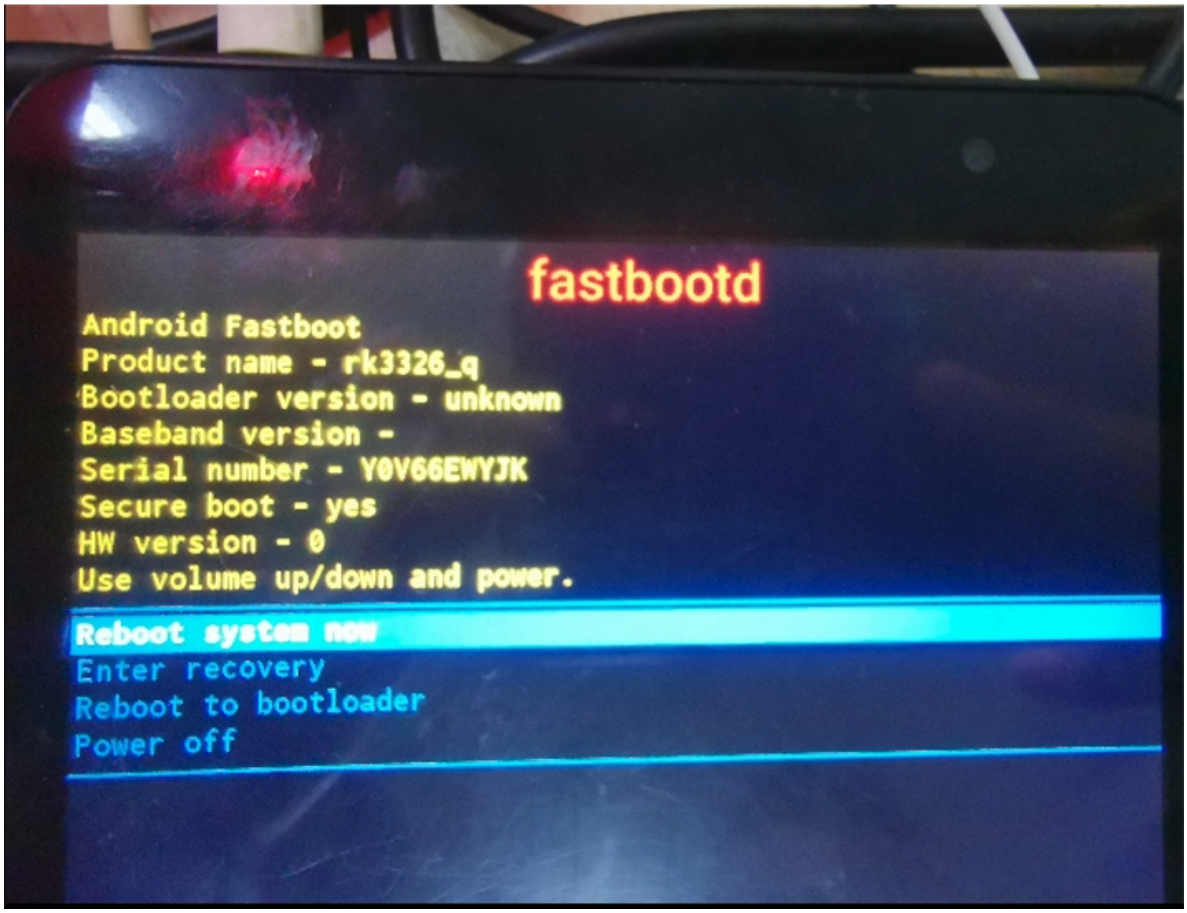
The new R device supports dynamic partitioning, and the system/vendor/odm/product/system_ext partitions have been removed. Please flash the super.img. For individual flashing of system/vendor/odm (corresponding img files can be found under out), use `fastbootd`. The adb and fastboot versions must both be the latest. The SDK provides a pre-compiled toolchain:

```
RKTools/linux/Linux_adb_fastboot (Linux_x86 version)
RKTools/Windows/adb_fastboot (Windows_x86 version)
```

- Flash dynamic partitions using commands:

```
adb reboot fastboot
fastboot flash vendor vendor.img
fastboot flash system system.img
fastboot flash odm odm.img
```

**Note**: After entering the fastbootd mode, the device information will be displayed on the screen, as shown in the figure:




**Note: Non-dynamic partitions use fastboot, please enter bootloader**

```
adb reboot bootloader
```

Methods for Flashing GSI:

- After confirming the device is unlocked, enter fastbootd mode, only the system.img from the GSI and the misc.img from the firmware need to be flashed. After flashing, the device will enter recovery mode to perform a factory reset. The complete flashing process is as follows:

1. Reboot to bootloader, Locked -> Device unlocked:

```
adb reboot bootloader
`fastboot oem at-unlock-vboot` ## For customers who have written AVB public
keys, please refer to the corresponding documentation for unlocking.
```

2. Restore factory settings, reboot into fastbootd:

```
fastboot flash misc misc.img
fastboot reboot fastboot  ## This will enter fastbootd mode.
```

3. Begin flashing the GSI

```
fastboot delete-logical-partition product  ## (optional) For devices with
limited partition space, execute this command first to delete the product
partition before flashing the GSI.
fastboot flash system system.img
fastboot reboot  ## After successful flashing, reboot
```

- Note: GSI can also be flashed using DSU (Dynamic System Updates). The Rockchip platform already supports DSU by default. Since this feature consumes significant memory, it is not recommended for devices with 1G DDR or lower. For DSU documentation and usage instructions, please refer to the official Android website:

https://source.android.com/devices/tech/ota/dynamic-system-updates

- Note 1: During the VTS test, the compiled boot-debug.img must be flashed to the boot partition.
- Note 2: The boot-debug.img is not required to be flashed during CTS-ON-GSI testing.
- Note 3: Use the GSI images officially released by Google ending with -signed during testing.

# 5. Using DTBO Features

Android 10.0 and above support the Device Tree Overlays feature, with the development process involving flashing the dtbo.img to achieve compatibility across multiple products, among other purposes.

Modification Method:

1. Find (or specify) the template file:

```
get_build_var PRODUCT_DTBO_TEMPLATE
```

For example:

```
PRODUCT_DTBO_TEMPLATE := $(LOCAL_PATH)/dt-
overlay.in(device/rockchip/rk388/rk3588_u/dt-overlay.in)
```

2. Add or modify the required nodes:

For example:

```
/dts-v1/;
/plugin/;

&chosen {
    bootargs_ext = "androidboot.boot_devices=${_boot_device}";
};

&firmware_android {
    vbmeta {
        status = "disabled";
    };
    fstab {
        status = "disabled";
    };
};

&reboot_mode {
    mode-bootloader = <0x5242C309>;
    mode-charge = <0x5242C30B>;
    mode-fastboot = <0x5242C303>;
    mode-loader = <0x5242C301>;
    mode-normal = <0x5242C300>;
    mode-recovery = <0x5242C303>;
};
```

**Note: When using dtbo, ensure that the dts contains an alias, otherwise the overlay will not succeed.**

# 6. Modifying the fstab File

1. Find (or specify) the template file:

```
get_build_var PRODUCT_FSTAB_TEMPLATE
```

For example:

```
PRODUCT_FSTAB_TEMPLATE := device/rockchip/common/scripts/fstab_tools/fstab.in
```

2. Modifications: adding partition mounting, modifying the swap_zram parameters, modifying the data partition format, etc.

# 7. Modifying parameter.txt

Android 14 adds a tool to generate parameter.txt, supporting compilation of parameter.txt based on configuration parameters. If no configuration template file is present, it will look for the modified parameter.txt file that has been added.

1. Find (or specify) the template file:

```
get_build_var PRODUCT_PARAMETER_TEMPLATE
```

For example:

```
PRODUCT_PARAMETER_TEMPLATE :=
device/rockchip/common/scripts/parameter_tools/parameter.in
```

2. Modify the configuration partition size (example):

```
BOARD_SUPER_PARTITION_SIZE := 2688548864
BOARD_DTBOIMG_PARTITION_SIZE := xxxx
BOARD_BOOTIMAGE_PARTITION_SIZE := xxxxx
BOARD_CACHEIMAGE_PARTITION_SIZE := xxxx
```

3. Do not use parameter generation tools:

Add a `parameter.txt` file to your `device` directory:

For example: device/rockchip/rk3326/rk3326_u/parameter.txt

4. Use the tool to generate `parameter.txt` (e.g.):

```
parameter_tools --input
device/rockchip/common/scripts/parameter_tools/parameter.in --firmware-version
16.0 --machine-model rk3326 --manufacturer rockchip --machine rk3326_u --
partition-list
uboot_a:4096K,trust_a:4M,misc:4M,dtbo_a:4M,vbmeta_a:4M,boot_a:33554432,backup:30
0M,security:4M,cache:300M,metadata:4096,frp:512K,super:2G --output
parameter_new.txt
```

# 8. Android14 SDK

Other parts of the documentation please refer to the developer guides in the Android14 SDK:
`RK3576_Developer_Guide_Android14_SDK_CN.pdf`
`Rockchip_Developer_Guide_Android14_SDK_CN.pdf`.

# 9. Appendix A Compiling and Development Environment Setup

## 9.1 Initializing a Build Environment

This section describes how to set up your local work environment to build the Android source files. You must use Linux or Mac OS; building under Windows is not currently supported.

For an overview of the entire code-review and code-update process, see Life of a Patch.

Note: All commands in this site are preceded by a dollar sign ($) to differentiate them from output or entries within files. You may use the Click to copy feature at the top right of each command box to copy all lines without the dollar signs or triple-click each line to copy it individually without the dollar sign.

## 9.2 Choosing a Branch

Some requirements for the build environment are determined by the version of the source code you plan to compile. For a full list of available branches, see Build Numbers. You can also choose to download and build the latest source code (called master), in which case you will simply omit the branch specification when you initialize the repository.

After you have selected a branch, follow the appropriate instructions below to set up your build environment.

## 9.3 Setting up a Linux build environment

These instructions apply to all branches, including master.

The Android build is routinely tested in house on recent versions of Ubuntu LTS (14.04) and Debian testing. Most other distributions should have the required build tools available.

For Gingerbread (2.3.x) and newer versions, including the master branch, a 64-bit environment is required. Older versions can be compiled on 32-bit systems.

Note: See Requirements for the complete list of hardware and software requirements, then follow the detailed instructions for Ubuntu and Mac OS below.

## 9.4 Installing the JDK

The master branch of Android in the Android Open Source Project (AOSP) comes with prebuilt versions of OpenJDK below prebuilts/jdk/ so no additional installation is required.

Older versions of Android require a separate installation of the JDK. On Ubuntu, use OpenJDK. See JDK Requirements for precise versions and the sections below for instructions.

**For Ubuntu >= 15.04**

Run the following:

```
sudo apt-get update
sudo apt-get install openjdk-8-jdk
```

**For Ubuntu LTS 14.04**

There are no available supported OpenJDK 8 packages for Ubuntu 14.04. The Ubuntu 15.04 OpenJDK 8 packages have been used successfully with Ubuntu 14.04. Newer package versions (e.g. those for 15.10, 16.04) were found not to work on 14.04 using the instructions below.

1. Download the .deb packages for 64-bit architecture from old-releases.ubuntu.com:

```
openjdk-8-jre-headless_8u45-b14-1_amd64.deb with SHA256
0f5aba8db39088283b51e00054813063173a4d8809f70033976f83e214ab56c0
openjdk-8-jre_8u45-b14-1_amd64.deb with SHA256
9ef76c4562d39432b69baf6c18f199707c5c56a5b4566847df908b7d74e15849
openjdk-8-jdk_8u45-b14-1_amd64.deb with SHA256
6e47215cf6205aa829e6a0a64985075bd29d1f428a4006a80c9db371c2fc3c4c
```

2. Optionally, confirm the checksums of the downloaded files against the SHA256 string listed with each package above. For example, with the sha256sum tool:

```
sha256sum {downloaded.deb file}
```

3. Install the packages:

```
sudo apt-get update
```

Run dpkg for each of the .deb files you downloaded. It may produce errors due to missing dependencies:

```
sudo dpkg -i {downloaded.deb file}
```

To fix missing dependencies:

```
sudo apt-get -f install
```

Update the default Java version - optional

Optionally, for the Ubuntu versions above update the default Java version by running:

```
sudo update-alternatives --config java
sudo update-alternatives --config javac
```

Note: If, during a build, you encounter version errors for Java, see Wrong Java version for likely causes and solutions.

**Installing required packages (Ubuntu 14.04)**

You will need a 64-bit version of Ubuntu. Ubuntu 14.04 is recommended.

```
sudo apt-get install git-core gnupg flex bison gperf build-essential zip curl
zlib1g-dev gcc-multilib g++-multilib libc6-dev-i386 lib32ncurses5-dev x11proto-
core-dev libx11-dev lib32z-dev ccache libgl1-mesa-dev libxml2-utils xsltproc
unzip python-pyelftools python3-pyelftools device-tree-compiler libfdt-dev
libfdt1 libssl-dev liblz4-tool python-dev
```

Note: To use SELinux tools for policy analysis, also install the python-networkx package. Note: If you are using LDAP and want to run ART host tests, also install the libnss-sss:i386 package.

## 9.5 Configuring USB Access

Under GNU/linux systems (and specifically under Ubuntu systems), regular users can't directly access USB devices by default. The system needs to be configured to allow such access.

The recommended approach is to create a file /etc/udev/rules.d/51-android.rules (as the root user) and to copy the following lines in it.  must be replaced by the actual username of the user who is authorized to access the phones over USB.

```
# adb protocol on passion (Rockchip products)
SUBSYSTEM=="usb", ATTR{idVendor}=="2207", ATTR{idProduct}=="0010", MODE="0600",
OWNER="<username>"
```

Those new rules take effect the next time a device is plugged in. It might therefore be necessary to unplug the device and plug it back into the computer.

This is known to work on both Ubuntu Hardy Heron (8.04.x LTS) and Lucid Lynx (10.04.x LTS). Other versions of Ubuntu or other variants of GNU/linux might require different configurations.

References : [http://source.android.com/source/initializing.html](http://source.android.com/source/initializing.html)

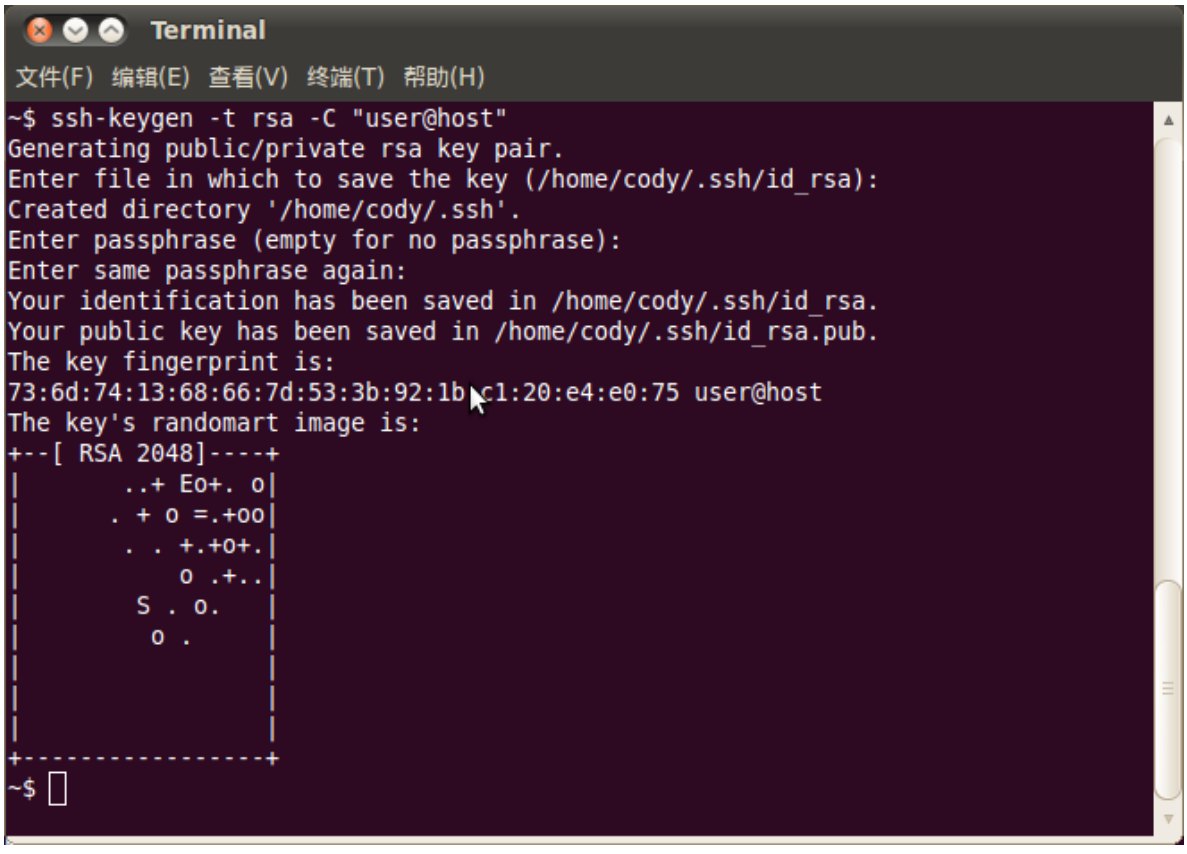
# 10. Appendix B SSH Public Key Operation Instructions

## 10.1 Appendix B-1 SSH public key generation

Use the following command to generate:

```
ssh-keygen -t rsa -C "user@host"
```

Please replace `user@host` with your email address.



Upon completion of the command execution, a key file will be generated in your directory.

```
~$ ls -l .ssh/
总用量 8
-rw------- 1 cody cody 1675 2012-10-15 11:38 id_rsa
-rw-r--r-- 1 cody cody  391 2012-10-15 11:38 id_rsa.pub
```

Securely store the generated private key file `id_rsa` and the password, and email `id_rsa.pub` to the administrator of the SDK distribution server.

## 10.2 Appendix B-2 Use key-chain to manage the key

We recommend using a simple tool such as Keychain to manage keys.

The specific usage method is as follows:

1. Install the keychain package:

```
$sudo aptitude install keychain
```

2. Configure the use of keys:

```
$vim ~/.bashrc
```

Add the following line:

```
eval `keychain --eval ~/.ssh/id_rsa`
```

In this context, id_rsa is the private key file name.

After the above configuration, re-login to the console will prompt for password input. Simply enter the password used during key generation. If no password was set, it can be left blank.

Additionally, avoid using sudo or root user unless you know how to handle it, as it may lead to permission and key management chaos.

## 10.3 Appendix B-3 Multiple devices use the same SSH public key

To use it on different machines, you can copy your SSH private key file `id_rsa` to the `~/.ssh/id_rsa` location on the target machine.

The following prompt will appear when an incorrect private key is used. Please ensure the correct private key is substituted.

```
~/tmp$ git clone git@172.16.10.211:rk292x/mid/4.1.1_r1
Initialized empty Git repository in /home/cody/tmp/4.1.1_r1/.git/
The authenticity of host '172.16.10.211 (172.16.10.211)' can't be established.
RSA key fingerprint is fe:36:dd:30:bb:83:73:e1:0b:df:90:e2:73:e4:61:46.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '172.16.10.211' (RSA) to the list of known hosts.
git@172.16.10.211's password:
```

After adding the correct private key, you can use git to clone the code, as shown in the figure below.

```
~$ cd tmp/
~/tmp$ git clone git@172.16.10.211:rk292x/mid/4.1.1_r1
Initialized empty Git repository in /home/cody/tmp/4.1.1_r1/.git/
The authenticity of host '172.16.10.211 (172.16.10.211)' can't be established.
RSA key fingerprint is fe:36:dd:30:bb:83:73:e1:0b:df:90:e2:73:e4:61:46.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '172.16.10.211' (RSA) to the list of known hosts.
remote: Counting objects: 237923, done.
remote: Compressing objects: 100% (168382/168382), done.
Receiving objects:   9% (21570/237923), 61.52 MiB | 11.14 MiB/s
```

Adding an SSH private key may result in the following prompt error.

```
Agent admitted failture to sign using the key
```

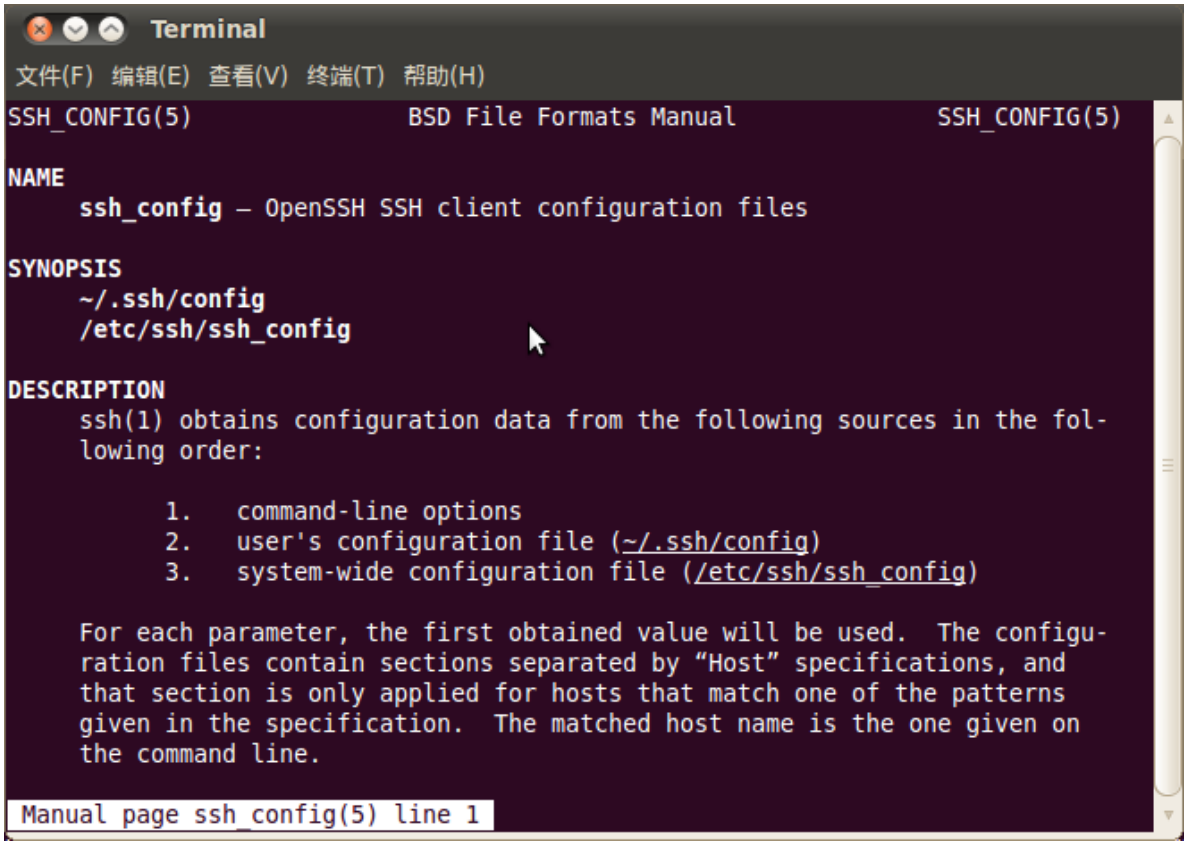
Enter the following command in the `console` to resolve it.

```
ssh-add ~/.ssh/id_rsa
```

## 10.4 Appendix B-4 Switching different SSH public keys on a single device

You can refer to the ssh_config documentation to configure SSH.

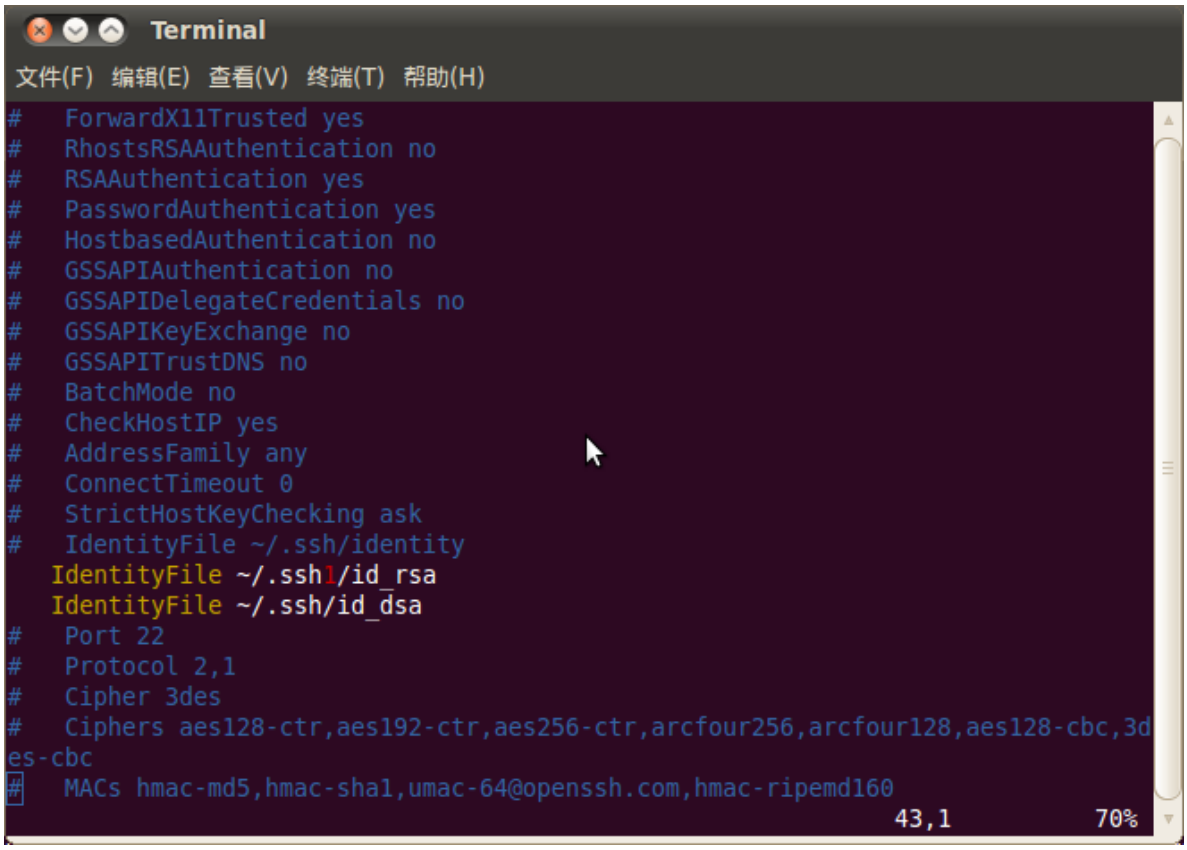
```
~$ man ssh_config
```




Use the following command to configure the SSH configuration for the current user.

```
~$ cp /etc/ssh/ssh_config ~/.ssh/config
~$ vi .ssh/config
```

As shown, use the file `~/.ssh1/id_rsa` from another directory as the authentication private key. This method allows switching between different keys.




## 10.5 Appendix B-5 Key Authority Management

The server can perform real-time monitoring of a key's download count, IP address, and other relevant information, and will disable the download permissions for the corresponding key if any anomalies are detected.

Please keep the private key file securely. Do not re-authorize its use by third parties.

## 10.6 Appendix B-6 Git Permission Application Instructions

Refer to the aforementioned section to generate the public key file, send an email to fae@rock-chips.com to apply for SDK code download access.